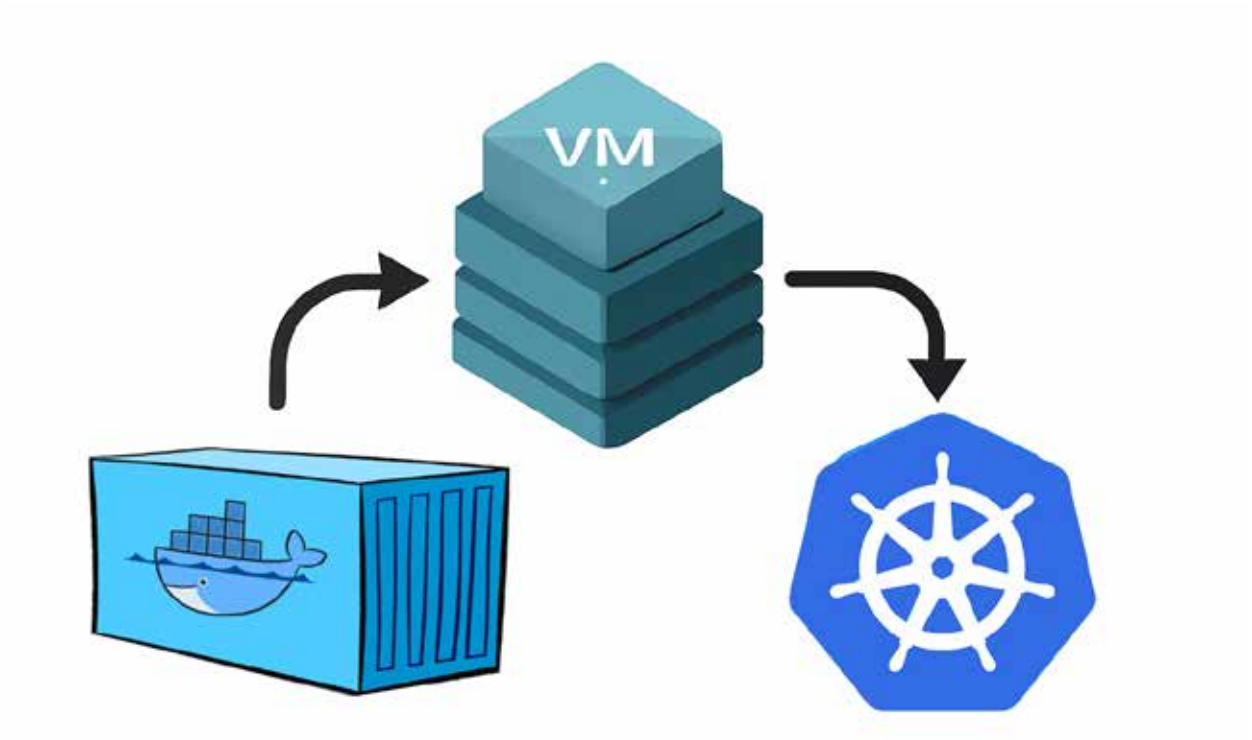
The Journey from VMs to Containers and Kubernetes

Let's walk through the journey from traditional VMs to containers and, finally, to Kubernetes. We'll keep things simple, explain the key concepts along the way, and share practical insights that can help you whether you're new to DevOps or just curious about what's powering today's cloud-native apps.



Over the past decade, the way we build and deploy software has changed dramatically. Not long ago, developers and IT teams relied heavily on traditional servers and virtual machines (VMs) to run applications. While those tools served us well, they often came with challenges—like high resource usage, complicated scaling, and slow deployment cycles.

Containers have become one of the most exciting shifts in modern software development. They offer a lightweight, flexible, and efficient way to package and run applications. Whether you're a solo developer working on side projects or part of a large team managing enterprise apps, containers help you move faster and more reliably.

But the story doesn't stop with containers. As applications grow and become more complex, we need ways to manage them at scale—and that's where Kubernetes steps in. Together, Docker and Kubernetes are helping developers rethink how software is built, shipped, and operated.

Virtual machines vs containers: Understanding the shift

Before we dive deeper into containers, let's take a moment to understand where we came from—virtual machines (VMs).

A virtual machine is like having a computer inside your computer. You can run an entire operating system (like Linux or Windows) inside a VM, even if your actual computer is running something else. VMs are great for isolation—they allow you to run different apps without them interfering with each other.

But there's a downside: they're heavy. Every VM needs its own full operating system, which means more memory, more disk space, and slower boot times. Running just a few